

Końcowa dokumentacja projektu

System wyznaczania współrzędnych dla wizualizacji grafów planarnych

Implementacja w języku Java z interfejsem Swing

Autorzy:

Krzysztof Wasilewski, Jakub Pietrzkiewicz

Data: 08.06.2026

Spis treści

1	Wprowadzenie	3
1.1	Cel projektu	3
2	Charakterystyka funkcjonalna aplikacji	4
2.1	Funkcje udostępniane użytkownikowi	4
2.2	Interfejs użytkownika	4
2.3	Przykładowe scenariusze użycia	5
3	Architektura oprogramowania	7
3.1	Podział na moduły	7
3.2	Diagram klas UML	8
3.3	Przepływ sterowania	9
4	Szczegółowy opis kodu	10
4.1	Moduł Main	10
4.2	Moduł Gui	10
4.3	Moduł GraphPanel	11
4.4	Moduł Graph	12
4.5	Moduł Node	14
4.6	Moduł Edge	14
4.7	Moduł AdjacencyList	15
4.8	Moduł Neighbor	16
4.9	Moduł Fruchterman	16
4.10	Moduł Tutte	18
4.11	Moduł ExitCodes	20
5	Specyfikacja wejścia i wyjścia	21
5.1	Wejście tekstowe (graf)	21
5.2	Wejście binarne (współrzędne)	21
5.3	Wyjście tekstowe	21
5.4	Wyjście binarne	22
6	Zakończenie dokumentacji projektu	23
6.1	Ograniczenia	23

1 Wprowadzenie

Projekt realizuje aplikację graficzną w języku Java z interfejsem użytkownika opartym na bibliotece Swing. Aplikacja umożliwia wizualizację grafów planarnych podanych jako listy krawędzi i wyznaczanie współrzędnych wierzchołków przy użyciu dwóch algorytmów:

- Fruchterman-Reingold (algorytm siłowy),
- Tutte embedding (osadzenie z relaksacją sąsiadów i ustalonym brzegiem).

Aplikacja obsługuje wczytywanie danych z formatów tekstowych i binarnych, zmianę widoku, edycję współrzędnych wierzchołków oraz eksport wyników. Kod ma budowę modułową z jasnym podziałem odpowiedzialności między komponenty. Dokument przedstawia:

- szczegółową analizę kodu i architektury,
- diagram klas UML,
- opis interfejsu użytkownika,
- formaty danych wejściowych i wyjściowych,
- dokumentację poszczególnych modułów i algorytmów,
- ograniczenia

1.1 Cel projektu

Celem projektu jest dostarczenie interaktywnego narzędzia umożliwiającego wczytywanie, wizualizację i edycję rozmieszczenia wierzchołków grafów planarnych. Główne założenia:

- intuicyjny interfejs graficzny (GUI) oparty na Swing,
- obsługa stałych formatów danych (tekstowy i binarny),
- możliwość wyboru algorytmu rozmieszczenia wierzchołków,
- interaktywna edycja współrzędnych poprzez manipulację myszą,
- nowoczesna architektura modułowa z wyraźnym podziałem odpowiedzialności.

2 Charakterystyka funkcjonalna aplikacji

2.1 Funkcje udostępniane użytkownikowi

Aplikacja Swing udostępnia następujące funkcje:

- wczytanie grafu z pliku tekstowego,
- wczytanie precomputed współrzędnych z pliku tekstowego,
- wczytanie precomputed współrzędnych z pliku binarnego,
- automatyczne dopasowanie widoku do okna,
- zoom in/out przy użyciu kółka myszy,
- przesuwanie widoku poprzez przeciąganie myszą,
- edycja współrzędnych wierzchołków w trybie edit mode,
- przełączanie wyświetlania etykiet wierzchołków,
- zapis współrzędnych do pliku tekstowego,
- zapis współrzędnych do pliku binarnego.

2.2 Interfejs użytkownika

Interfejs składa się z trzech głównych komponentów:

1. Pasek menu (`createMenuBar()`)

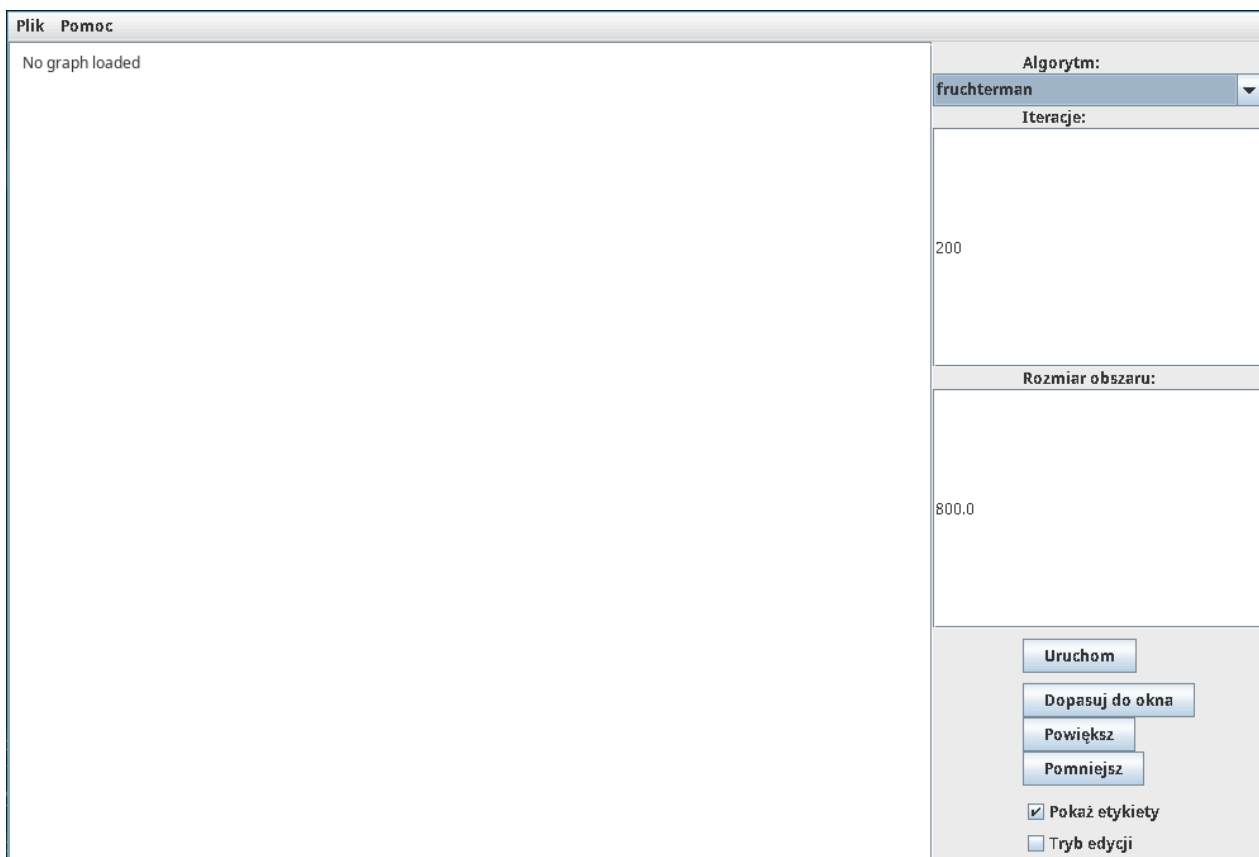
- Menu „Plik”:
 - „Wczytaj graf (tekstowy)” - otwiera dialog wyboru pliku z grafem,
 - „Wczytaj współrzędne (tekstowe)” - wczytuje współrzędne z pliku tekstowego,
 - „Wczytaj współrzędne (binarne)” - wczytuje współrzędne z pliku binarnego,
 - „Zapisz współrzędne (tekstowy)” - eksportuje współrzędne do pliku tekstowego,
 - „Zapisz współrzędne (binarny)” - eksportuje współrzędne do pliku binarnego.

2. Panel graficzny (komponenta `GraphPanel`)

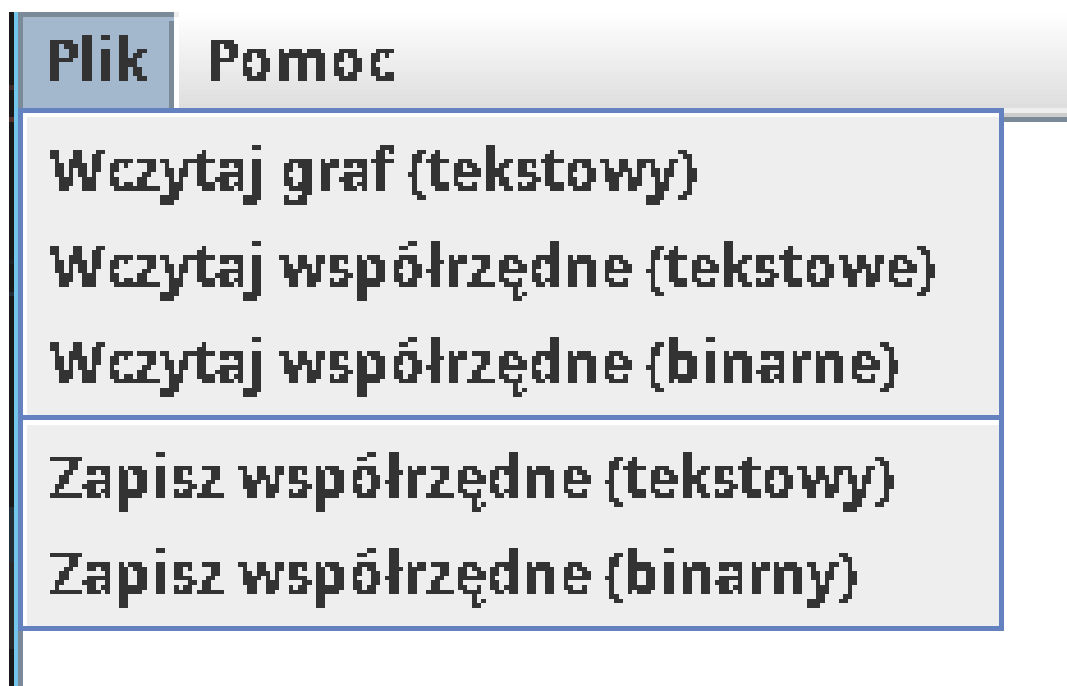
- renderuje graf na kanwie,
- obsługuje zdarzenia myszy (zoom, pan, edycja),
- wyświetla wierzchołki jako koła i krawędzie jako linie,
- opcjonalnie wyświetla etykiety wierzchołków.

3. Panel narzędziowy (po prawej stronie, `createToolPanel()`)

- suwaki do zmiany skali zoom,
- przyciski do resetowania widoku i dopasowania do okna,
- pole wyboru trybu edit,
- checkbox do przełączania wyświetlania etykiet.



Rysunek 1: Widok głównego interfejsu aplikacji (GUI)



Rysunek 2: Menu wczytywania i zapisu plików

2.3 Przykładowe scenariusze użycia

Scenariusz 1: Wczytanie grafu z wizualizacją

1. Uruchomienie aplikacji,
2. Wczytanie grafu z pliku (Plik → Wczytaj graf (tekstowy)),
3. Aplikacja automatycznie dopasowuje widok do okna.

Scenariusz 2: Wczytanie i edycja współrzędnych

1. Wczytanie grafu,
2. Wczytanie precomputed współrzędnych z pliku C (Plik → Wczytaj współrzędne (tekstowe)),
3. Aktywowanie trybu edit (Panel narzędziowy → Tryb edycji),
4. Redagowanie współrzędnych wierzchołków poprzez przeciąganie,
5. Zapis wyników (Plik → Zapisz współrzędne (tekstowy)).

Scenariusz 3: Zoom i pan

1. Wczytanie grafu,
2. Przewijanie kółka myszy - zoom in/out,
3. Przeciąganie myszą (przycisk lewy) - przesunięcie widoku,
4. Reset widoku (Panel narzędziowy - przycisk „Dopasuj do okna”).

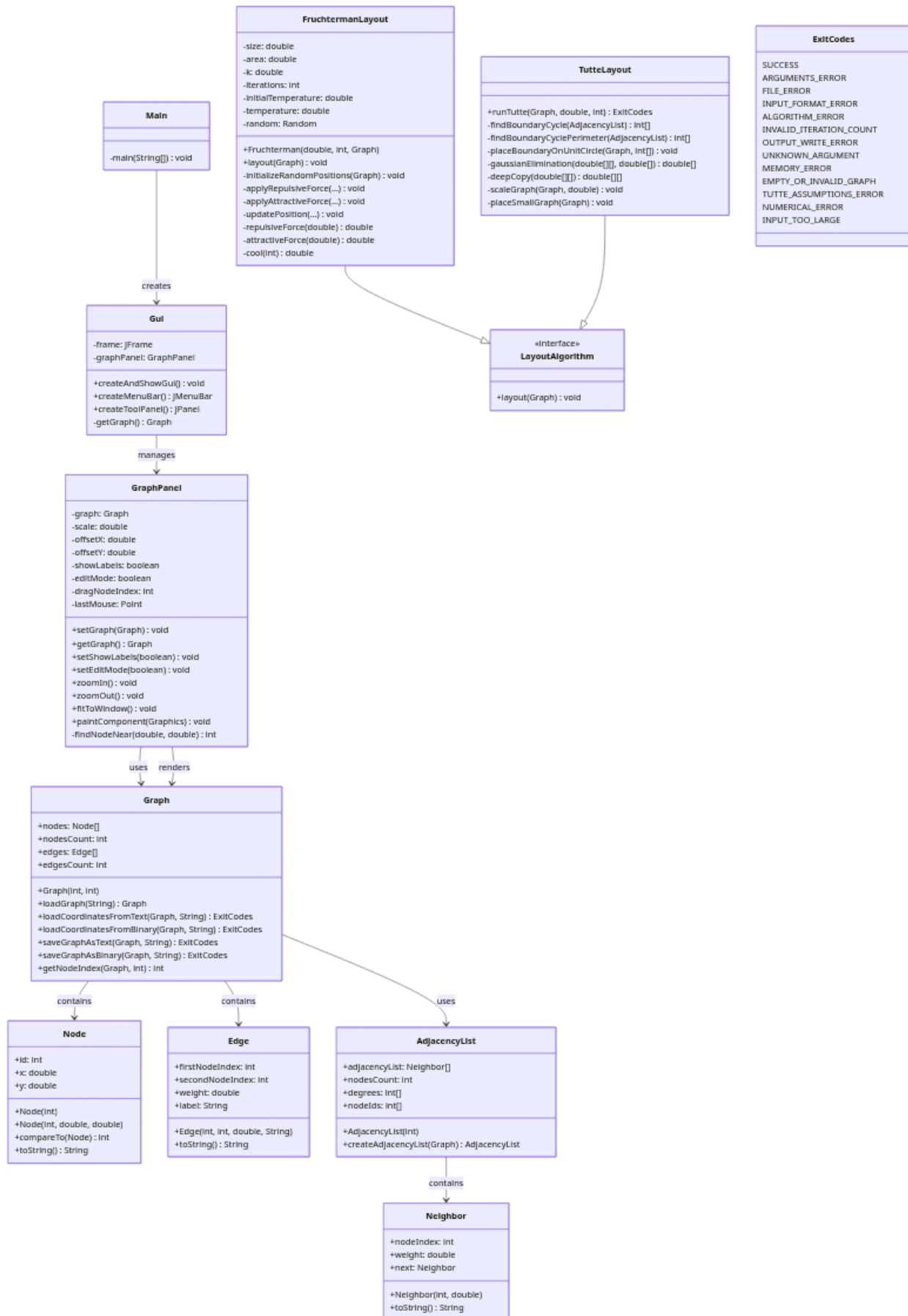
3 Architektura oprogramowania

3.1 Podział na moduły

Projekt Java zawiera następujące główne moduły:

- Main - punkt wejścia aplikacji, inicjalizacja GUI na Event Dispatch Thread,
- Gui - zarządzanie oknem głównym, menu, panelem narzędziowym,
- GraphPanel - komponenta Swing do renderowania i interakcji z grafem,
- Graph - struktura danych i zarządzanie grafem, I/O,
- Node - reprezentacja wierzchołka (id, x, y),
- Edge - reprezentacja krawędzi (firstNodeIndex, secondNodeIndex, weight, label),
- AdjacencyList - struktura do reprezentacji listy sąsiadów,
- Neighbor - element listy sąsiadów,
- Fruchterman - implementacja algorytmu Fruchterman-Reingold,
- Tutte - implementacja algorytmu Tutte embedding,
- ExitCodes - enum kodów powrotu/statusów.

3.2 Diagram klas UML



Rysunek 3: Diagram klas UML

3.3 Przepływ sterowania

1. Uruchomienie: `Main.main()` tworzy instancję `Gui` na `Event Dispatch Thread`,
2. Inicjalizacja GUI: `Gui.createAndShowGui()` tworzy okno z menu, `GraphPanel` i panelem narzędziowym,
3. Wczytanie danych: akcje menu wywołują `Graph.loadGraph()` lub `Graph.loadCoordinatesFrom*()`,
4. Renderowanie: `GraphPanel.paintComponent()` rysuje graf na kanwie,
5. Interakcja: eventy myszy w `GraphPanel` obsługują zoom, pan i edycję,
6. Eksport: akcje menu wywołują `Graph.saveGraph*()`.

4 Szczegółowy opis kodu

4.1 Moduł Main

Rola: punkt wejścia aplikacji i inicjalizacja GUI na wątku Event Dispatch.

Kod:

```
package pl.graph;

import javax.swing.SwingUtilities;

public class Main {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Gui().createAndShowGui());
    }
}
```

Wyjaśnienie:

- `SwingUtilities.invokeLater()` gwarantuje, że GUI jest tworzony na wątku Event Dispatch Thread,
- `new Gui().createAndShowGui()` inicjalizuje wszystkie komponenty interfejsu.

4.2 Moduł Gui

Rola: zarządzanie głównym oknem, menu i panelem narzędziowym.

Opis pól:

- `frame`: `JFrame` - główne okno aplikacji,
- `graphPanel`: `GraphPanel` - komponenta do rysowania grafu.

Główne metody:

- `createAndShowGui()` - tworzy i wyświetla okno główne,
- `createMenuBar()` - buduje pasek menu z akcjami do wczytania i zapisu,
- `createToolPanel()` - buduje panel narzędziowy po prawej stronie,
- `getGraph()` - zwraca aktualny graf z panelu.

Fragment kodujący wczytanie grafu:

```
JMenuItem loadTxt = new JMenuItem(new AbstractAction("Wczytaj graf (tekstowy)") {
    @Override
    public void actionPerformed(ActionEvent e) {
        JFileChooser fc = new JFileChooser();
        if (fc.showOpenDialog(frame) == JFileChooser.APPROVE_OPTION) {
            File f = fc.getSelectedFile();
            try {
                Graph g = Graph.loadGraph(f.getAbsolutePath());
                graphPanel.setGraph(g);
            } catch (IOException | IllegalArgumentException ex) {
                JOptionPane.showMessageDialog(frame,
                    "Błąd wczytywania: " + ex.getMessage(),
                    "Błąd", JOptionPane.ERROR_MESSAGE);
            }
        }
    }
});
```

Obsługa błędów: wyświetlenie okna dialogowego z komunikatem o błędzie.

4.3 Moduł GraphPanel

Rola: Komponenta Swing do rysowania grafu i obsługi interakcji myszy.

Opis pól:

- graph: Graph - wczytany graf,
- scale: double - początkowa skala (zoom),
- offsetX, offsetY: double - przesunięcie widoku,
- showLabels: boolean - flaga wyświetlania etykiet,
- editMode: boolean - tryb edycji współrzędnych,
- dragNodeIndex: int - indeks przeciąganego wierzchołka,
- lastMouse: Point - ostatnia pozycja myszy.

Główne metody:

- setGraph(Graph) - ustawia graf i dopasowuje widok,
- paintComponent(Graphics) - rysuje wierzchołki i krawędzie,
- zoomIn(), zoomOut() - sterowanie skalą,
- fitToWindow() - automatyczne dopasowanie widoku,
- findNodeNear(double, double): int - znalezienie wierzchołka położonego blisko punktu.

Fragment kodujący obsługę myszy i przesuwania:

```
MouseAdapter ma = new MouseAdapter() {
    @Override
    public void mouseDragged(MouseEvent e) {
        if (dragNodeIndex >= 0 && graph != null) {
            Node n = graph.nodes[dragNodeIndex];
            Point p = e.getPoint();
            double gx = (p.x - offsetX) / scale;
            double gy = (p.y - offsetY) / scale;
            n.x = gx;
            n.y = gy;
            repaint();
        } else if (lastMouse != null) {
            Point p = e.getPoint();
            offsetX += p.x - lastMouse.x;
            offsetY += p.y - lastMouse.y;
            lastMouse = p;
            repaint();
        }
    }

    @Override
    public void mouseWheelMoved(MouseWheelEvent e) {
        int notches = e.getWheelRotation();
        if (notches < 0) zoomIn();
        else zoomOut();
    }
};
```

Wyjaśnienie:

- Jeśli editMode jest aktywny i mamy wciśnięty wierzchołek → edytuj jego współrzędne,
- w przeciwnym razie → przesuwać widok,
- kółko myszy zmienia skalę.

4.4 Moduł Graph

Rola: struktura danych dla grafu, wczytywanie i zapis danych.

Opis pól:

- nodes: Node[] - tablica wierzchołków (posortowana po ID),
- nodesCount: int - liczba wierzchołków,
- edges: Edge[] - tablica krawędzi,
- edgesCount: int - liczba krawędzi.

Główne metody statyczne:

- loadGraph(String): Graph - wczytuje graf z pliku tekstowego,
- loadCoordinatesFromText(Graph, String): ExitCodes - wczytuje współrzędne z formatu tekstowego,
- loadCoordinatesFromBinary(Graph, String): ExitCodes - wczytuje współrzędne z formatu binarnego,
- saveGraphAsText(Graph, String): ExitCodes - zapisuje współrzędne w formacie tekstowym,
- saveGraphAsBinary(Graph, String): ExitCodes - zapisuje współrzędne w formacie binarnym,
- getNodeIndex(Graph, int): int - wyszukuje indeks wierzchołka po ID (wyszukiwanie binarne).

Fragment kodujący wczytanie grafu:

```
public static Graph loadGraph(String path) throws IOException {
    try (BufferedReader reader = new BufferedReader(new FileReader(path))) {
        String line;
        Set<Integer> uniqueNodeIds = new LinkedHashSet<>();
        List<ParsedEdge> edges = new ArrayList<>();

        // Przejście po pliku: zbieramy unikalne ID węzłów oraz listę krawędzi
        while ((line = reader.readLine()) != null) {
            line = line.trim();

            // Pomijanie białych linii i komentarzy
            if (line.isEmpty() || line.startsWith("#")) {
                continue;
            }

            int commentIndex = line.indexOf('#');
            if (commentIndex >= 0) {
                line = line.substring(0, commentIndex).trim();
            }

            if (line.isEmpty()) {
                continue;
            }

            String[] parts = line.split("\\s+");
            if (parts.length < 3) {
                continue;
            }

            try {
                String label = parts[0];
                if (label.length() > 32) {
                    label = label.substring(0, 32);
                }

                int firstNode = Integer.parseInt(parts[1]);
```

```

        int secondNode = Integer.parseInt(parts[2]);

        double weight = 1.0;
        if (parts.length >= 4) {
            try {
                weight = Double.parseDouble(parts[3]);
            } catch (NumberFormatException ignored) {
                // domyślna waga
            }
        }

        uniqueNodeIds.add(firstNode);
        uniqueNodeIds.add(secondNode);
        edges.add(new ParsedEdge(label, firstNode, secondNode, weight));
    } catch (NumberFormatException e) {
        // ignoruj linię
    }
}

// Tworzenie grafu i mapowanie ID na indeksy
int nodesCount = uniqueNodeIds.size();
int edgesCount = edges.size();
Graph graph = new Graph(nodesCount, edgesCount);

int idx = 0;
for (int nodeId : uniqueNodeIds) {
    graph.nodes[idx] = new Node(nodeId);
    idx++;
}
Arrays.sort(graph.nodes); // Sortowanie po ID

// ... mapowanie krawędzi na indeksy
return graph;
}
}

```

Wyjaśnienie:

- wczytanie jest jedno przebiegowe,
- ignorowanie komentarzy (linie zaczynające się od #),
- automatyczne sortowanie wierzchołków po ID,
- mapowanie identyfikatorów na indeksy w tablicy.

Fragment kodujący zapis do formatu binarnego:

```

public static ExitCodes saveGraphAsBinary(Graph graph, String path) {
    try (DataOutputStream dos = new DataOutputStream(
        new BufferedOutputStream(new FileOutputStream(path)))) {
        dos.writeInt(Integer.reverseBytes(graph.nodesCount));
        for (int i = 0; i < graph.nodesCount; i++) {
            dos.writeInt(Integer.reverseBytes(graph.nodes[i].id));
            dos.writeLong(Long.reverseBytes(
                Double.doubleToLongBits(graph.nodes[i].x)));
            dos.writeLong(Long.reverseBytes(
                Double.doubleToLongBits(graph.nodes[i].y)));
        }
        return ExitCodes.SUCCESS;
    } catch (IOException e) {
        System.err.println("Error: Cannot write to file: " + path);
        return ExitCodes.OUTPUT_WRITE_ERROR;
    }
}

```

```
    }  
}
```

Wyjaśnienie:

- każdy wierzchołek zapisywany jako 20 bajtów: int id (4), double x (8), double y (8),
- reverseBytes() zapewnia kompatybilność endianności,
- stosowana jest buforyzacja (BufferedOutputStream).

4.5 Moduł Node

Rola: reprezentacja wierzchołka.

Kod:

```
public class Node implements Comparable<Node> {  
    public int id;  
    public double x;  
    public double y;  
  
    public Node(int id) {  
        this.id = id;  
        this.x = 0.0;  
        this.y = 0.0;  
    }  
  
    public Node(int id, double x, double y) {  
        this.id = id;  
        this.x = x;  
        this.y = y;  
    }  
  
    @Override  
    public int compareTo(Node other) {  
        return Integer.compare(this.id, other.id);  
    }  
}
```

Wyjaśnienie:

- Comparable<Node> pozwala na sortowanie wierzchołków po ID,
- pola x, y są publiczne, co ułatwia szybki dostęp i modyfikację.

4.6 Moduł Edge

Rola: reprezentacja krawędzi.

Kod:

```
public class Edge {  
    public int firstNodeIndex;  
    public int secondNodeIndex;  
    public double weight;  
    public String label;  
  
    public Edge(int firstNodeIndex, int secondNodeIndex,  
                double weight, String label) {  
        this.firstNodeIndex = firstNodeIndex;  
        this.secondNodeIndex = secondNodeIndex;  
        this.weight = weight;  
        this.label = label != null ? label : "";  
    }  
}
```

```
}  
}
```

Wyjaśnienie:

- przechowuje indeksy wierzchołków w tablicy, a nie referencje (szybszy dostęp),
- waga i etykieta są opcjonalne.

4.7 Moduł AdjacencyList

Rola: reprezentacja listy sąsiadów dla grafu.

Kod:

```
public class AdjacencyList {  
    public Neighbor[] adjacencyList;  
    public int nodesCount;  
    public int[] degrees;  
    public int[] nodeIds;  
  
    public AdjacencyList(int nodesCount) {  
        this.nodesCount = nodesCount;  
        this.adjacencyList = new Neighbor[nodesCount];  
        this.degrees = new int[nodesCount];  
        this.nodeIds = new int[nodesCount];  
    }  
  
    public static AdjacencyList createAdjacencyList(Graph graph) {  
        AdjacencyList adjList = new AdjacencyList(graph.nodesCount);  
  
        for (int i = 0; i < graph.nodesCount; i++) {  
            adjList.nodeIds[i] = graph.nodes[i].id;  
        }  
  
        // Dla każdej krawędzi dodajemy obie relacje  
        for (int i = 0; i < graph.edgesCount; i++) {  
            int firstIndex = graph.edges[i].firstNodeIndex;  
            int secondIndex = graph.edges[i].secondNodeIndex;  
            double weight = graph.edges[i].weight;  
  
            Neighbor neighbor1 = new Neighbor(secondIndex, weight);  
            neighbor1.next = adjList.adjacencyList[firstIndex];  
            adjList.adjacencyList[firstIndex] = neighbor1;  
  
            Neighbor neighbor2 = new Neighbor(firstIndex, weight);  
            neighbor2.next = adjList.adjacencyList[secondIndex];  
            adjList.adjacencyList[secondIndex] = neighbor2;  
  
            adjList.degrees[firstIndex]++;  
            adjList.degrees[secondIndex]++;  
        }  
  
        return adjList;  
    }  
}
```

Wyjaśnienie:

- lista sąsiadów przechowywana jako tablica list jednokierunkowych,
- dla każdej krawędzi dodawane są dwie relacje (graf nieskierowany).

4.8 Moduł Neighbor

Rola: element listy sąsiadów.

Kod:

```
public class Neighbor {
    public int nodeIndex;
    public double weight;
    public Neighbor next;

    public Neighbor(int nodeIndex, double weight) {
        this.nodeIndex = nodeIndex;
        this.weight = weight;
        this.next = null;
    }
}
```

4.9 Moduł Fruchterman

Rola: implementacja algorytmu Fruchterman-Reingold.

Opis pól:

- size: double - rozmiar obszaru layoutu,
- area: double - pole obszaru layoutu (size^2),
- k: double - stała odpychania: $k = \sqrt{\frac{\text{area}}{\max(1, \text{nodeCount})}}$,
- iterations: int - liczba iteracji,
- initialTemperature: double - temperatura początkowa: $T_0 = \frac{\text{size}}{10}$,
- temperature: double - temperatura aktualna,
- random: Random - generator liczb losowych.

Główne metody:

- layout(Graph) - główny algorytm,
- initializeRandomPositions(Graph) - losowe rozmieszczenie wierzchołków,
- applyRepulsiveForce() - siła odpychająca między parami wierzchołków,
- applyAttractiveForce() - siła przyciągająca dla każdej krawędzi,
- updatePosition() - aktualizacja pozycji wierzchołka,
- repulsiveForce(double) - wzór siły odpychającej,
- attractiveForce(double) - wzór siły przyciągającej,
- cool(int) - chłodzenie (zmniejszanie temperatury).

Algorytm:

1. Inicjalizacja losowych pozycji wierzchołków w obszarze $[0, \text{size}] \times [0, \text{size}]$,
2. Dla każdej iteracji: a. Resetowanie przemieszczeń, b. Dla każdej pary wierzchołków: obliczenie i zastosowanie siły odpychającej, c. Dla każdej krawędzi: obliczenie i zastosowanie siły przyciągającej, d. Aktualizacja pozycji wierzchołków z ograniczeniem temperatury, e. Chłodzenie (zmniejszenie temperatury).

Fragment kodujący główną pętlę:

```
public void layout(Graph graph) {
    if (graph == null || graph.nodesCount <= 0) {
        return;
    }

    temperature = initialTemperature;
    initializeRandomPositions(graph);
```



```

double[] displacementX = new double[graph.nodesCount];
double[] displacementY = new double[graph.nodesCount];

for (int iteration = 0; iteration < iterations; iteration++) {
    // Resetowanie przemieszczeń
    for (int i = 0; i < graph.nodesCount; i++) {
        displacementX[i] = 0.0;
        displacementY[i] = 0.0;
    }

    // Siły odpychające
    for (int i = 0; i < graph.nodesCount; i++) {
        Node nodeA = graph.nodes[i];
        for (int j = i + 1; j < graph.nodesCount; j++) {
            Node nodeB = graph.nodes[j];
            applyRepulsiveForce(nodeA, nodeB, displacementX,
                               displacementY, i, j);
        }
    }

    // Siły przyciągające
    for (int i = 0; i < graph.edgesCount; i++) {
        applyAttractiveForce(graph, graph.edges[i],
                             displacementX, displacementY);
    }

    // Aktualizacja pozycji
    for (int i = 0; i < graph.nodesCount; i++) {
        updatePosition(graph.nodes[i], displacementX[i],
                       displacementY[i]);
    }

    temperature = cool(iteration);
}
}

```

Fragment kodujący siłę odpychającą:

```

private void applyRepulsiveForce(Node a, Node b,
                                double[] displacementX,
                                double[] displacementY,
                                int indexA, int indexB) {

    double dx = a.x - b.x;
    double dy = a.y - b.y;
    double distance = Math.hypot(dx, dy);

    if (distance < 0.01) {
        dx = (random.nextDouble() - 0.5) * 0.1;
        dy = (random.nextDouble() - 0.5) * 0.1;
        distance = Math.hypot(dx, dy);
    }

    double force = repulsiveForce(distance);
    double deltaX = (dx / distance) * force;
    double deltaY = (dy / distance) * force;

    displacementX[indexA] += deltaX;
    displacementY[indexA] += deltaY;
    displacementX[indexB] -= deltaX;
}

```

```

displacementY[indexB] -= deltaY;
}

```

Wyjaśnienie wzorów:

- Siła odpychająca: $F_{r(d)} = \frac{k^2}{d}$,
- Siła przyciągająca: $F_{a(d)} = \frac{d^2}{k}$,
- Przesunięcie wierzchołka jest ograniczane przez temperaturę: $\Delta = F \cdot \min(|F|, T)$,
- Temperatura maleje liniowo w każdej iteracji: $T_i = \text{initialTemperature} \cdot \left(1 - \frac{i}{\text{iterations}}\right)$.

4.10 Moduł Tutte

Rola: implementacja algorytmu Tutte embedding (osadzenie sprężynowe z ustalone brzegiem).

Główne metody statyczne:

- runTutte(Graph, double, int): ExitCodes - główny algorytm,
- findBoundaryCycle(AdjacencyList): int[] - znalezienie cyklu brzegowego,
- findBoundaryCyclePerimeter(AdjacencyList): int[] - heurystyka znalezienia obwodu,
- placeBoundaryOnUnitCircle(Graph, int[]): void - rozmieszczenie brzegu na okręgu jednostkowym,
- gaussianElimination(double[][][], double[]): double[] - rozwiązanie układu liniowego,
 - scaleGraph(Graph, double): void - skalowanie grafu do obszaru $[0, \text{size}] \times [0, \text{size}]$.

Algorytm:

1. Znalezienie cyklu brzegowego grafu (heurystyka perimetru lub DFS),
2. Rozmieszczenie wierzchołków brzegowych na okręgu jednostkowym,
3. Dla każdego wierzchołka wewnętrznego: $v = \frac{1}{\deg(v)} \cdot \sum_{u \in N(v)} u$,
4. Rozwiązanie układu liniowego $L_I x = b_x$ i $L_I y = b_y$ (gdzie L_I to zredukowana macierz Laplace'a),
5. Skalowanie wyniku do obszaru $[0, \text{size}] \times [0, \text{size}]$.

Fragment kodujący budowę macierzy Laplace'a:

```

double[][] A = new double[internalCount][internalCount];
double[] bx = new double[internalCount];
double[] by = new double[internalCount];

for (int v = 0; v < graph.nodesCount; v++) {
    if (isBoundary[v]) {
        continue;
    }
    int row = internalIndex[v];

    int degree = 0;
    for (Neighbor neighbor = adjList.adjacencyList[v];
         neighbor != null; neighbor = neighbor.next) {
        int u = neighbor.nodeIndex;
        degree++;
        if (isBoundary[u]) {
            bx[row] += graph.nodes[u].x;
            by[row] += graph.nodes[u].y;
        } else {
            int col = internalIndex[u];
            if (col >= 0) {
                A[row][col] -= 1.0;
            }
        }
    }
}

```

```

    A[row][row] += degree;
}

```

Wyjaśnienie:

- Macierz A to zredukowana macierz Laplace'a (tylko dla wierzchołków wewnętrznych),
- wektor b zawiera sumy współrzędnych wierzchołków brzegowych.

Fragment kodujący eliminację Gaussa:

```

private static double[] gaussianElimination(double[][] A,
                                           double[] b) {
    int n = A.length;

    // Forward elimination with partial pivoting
    for (int col = 0; col < n; col++) {
        int pivotRow = col;
        for (int row = col + 1; row < n; row++) {
            if (Math.abs(A[row][col]) > Math.abs(A[pivotRow][col])) {
                pivotRow = row;
            }
        }

        // Swap rows
        double[] temp = A[col];
        A[col] = A[pivotRow];
        A[pivotRow] = temp;

        double bTemp = b[col];
        b[col] = b[pivotRow];
        b[pivotRow] = bTemp;

        // Check for singular matrix
        if (Math.abs(A[col][col]) < 1e-10) {
            return null;
        }

        // Eliminate column
        for (int row = col + 1; row < n; row++) {
            double factor = A[row][col] / A[col][col];
            for (int c = col; c < n; c++) {
                A[row][c] -= factor * A[col][c];
            }
            b[row] -= factor * b[col];
        }
    }

    // Back substitution
    double[] x = new double[n];
    for (int i = n - 1; i >= 0; i--) {
        x[i] = b[i];
        for (int j = i + 1; j < n; j++) {
            x[i] -= A[i][j] * x[j];
        }
        x[i] /= A[i][i];
    }

    return x;
}

```

4.11 Moduł ExitCodes

Rola: enumeru statusów/kodów powrotu.

Kod:

```
public enum ExitCodes {
    SUCCESS(0),
    ARGUMENTS_ERROR(1),
    FILE_ERROR(2),
    INPUT_FORMAT_ERROR(3),
    ALGORITHM_ERROR(4),
    INVALID_ITERATION_COUNT(5),
    OUTPUT_WRITE_ERROR(6),
    UNKNOWN_ARGUMENT(7),
    MEMORY_ERROR(8),
    EMPTY_OR_INVALID_GRAPH(9),
    TUTTE_ASSUMPTIONS_ERROR(10),
    NUMERICAL_ERROR(11),
    INPUT_TOO_LARGE(12);

    private final int code;

    ExitCodes(int code) {
        this.code = code;
    }

    public int code() {
        return code;
    }
}
```

5 Specyfikacja wejścia i wyjścia

5.1 Wejście tekstowe (graf)

Plik wejściowy opisuje graf jako listę krawędzi, po jednej na linię:

```
<etykieta> <wierzchołek_A> <wierzchołek_B> [waga]
```

Przykład:

```
AB 1 2 1.0
BC 2 3 1.0
CA 3 1 1.2
DB 4 2 1.407
```

Zasady parsowania:

- parser akceptuje białe znaki (spacja, tab, CR, LF),
- parser ignoruje całe linie komentarzy zaczynające się od #,
- parser toleruje komentarze inline po rekordzie,
- etykieta jest ograniczona do 32 znaków (długości Stringa Java),
- waga jest opcjonalna, domyślnie równa 1.0,
- wierzchołki są identyfikowane liczbami całkowitymi,
- identyfikatory muszą być nieujemne.

5.2 Wejście binarne (współrzędne)

Plik binarny służy do wczytania współrzędnych do już istniejącego grafu. Ma on postać zgodną z `saveGraphAsBinary()`:

1. 4 bajty: liczba wierzchołków `nodesCount`, zapisana jako `int` w kolejności little-endian,
2. dla każdego wierzchołka:
 - 4 bajty: `id` (`int`, little-endian),
 - 8 bajtów: `x` (`double`, little-endian),
 - 8 bajtów: `y` (`double`, little-endian).

Wczytywanie odbywa się przez dopasowanie identyfikatorów do istniejących wierzchołków grafu. Plik nie tworzy nowego grafu, tylko aktualizuje współrzędne tych wierzchołków, które uda się odnaleźć po `id`.

5.3 Wyjście tekstowe

Format:

```
<wierzchołek> <współrzędna_x> <współrzędna_y>
```

Przykład:

```
1 0.000000 0.000000
2 1000.000000 0.000000
3 500.000000 866.000000
4 300.000000 500.000000
```

Precyzja: 6 miejsc po przecinku (ustawienie `Locale.ROOT` dla konsystencji między platformami).

5.4 Wyjście binarne

Plik binarny rozpoczyna się od 4-bajtowego nagłówka z liczbą wierzchołków `nodesCount` zapisaną jako `int` w kolejności little-endian. Następnie zapisywane są rekordy dla kolejnych wierzchołków:

- 4 bajty: `int id`,
- 8 bajtów: `double x`,
- 8 bajtów: `double y`.

Uwagi:

- kolejność rekordów odpowiada kolejności w tablicy `nodes` (po sortowaniu po `id`),
- `reverseBytes()` zapewnia zapis i odczyt w little-endian,
- ten sam format jest używany przez zapis i odczyt współrzędnych,
- stosowana jest buforyzacja dla wydajności.

6 Zakończenie dokumentacji projektu

6.1 Ograniczenia

1. Rozmiar grafu

- Liczba wierzchołków: zależy od dostępnej pamięci JVM (zwykle do 100k),
- Liczba krawędzi: analogicznie,
- Dla bardzo dużych grafów (powyżej 10k wierzchołków) wydajność staje się problemem.

2. Algorytmy

- Fruchterman: $O(n^2)$ iteracji) - wolny dla dużych grafów,
- Tutte: wymaga spójnego grafu i możliwości znalezienia cyklu brzegowego,

3. Interfejs

- Brak eksportowania grafu do innego formatu (np. SVG, PDF),
- brak undo/redo dla edycji,